DealLock

A Milestone-Based Escrow dApp for Digital Freelance Deliverables

FTE4312 — Course Project Final Report

April 2026

Ho Dang Dung (Student ID: 325040027)

Taisiia Ibragimova (Student ID: 325040033)

Abstract

DealLock is a browser-native dApp for milestone-based escrow in structured digital freelance agreements. One Solidity contract represents one agreement between a client and a provider, holding native ETH and enforcing acceptance, exact funding, sequential submission, approval, and refund of expired pending milestones. The React/Vite frontend uses wagmi and viem to deploy and operate contracts on Sepolia, while optional titles, notes, and agreement history stay in browser `localStorage`. Evaluation is based on an eleven-case Hardhat test suite covering the reachable contract state transitions and a Sepolia-oriented demo workflow. The project is intentionally scoped as an MVP: it enforces payment flow, not subjective quality judgement or arbitration.

Contents

1 Introduction **2**

2 Background **3**

2.1 Escrow in digital service delivery 3

2.2 Smart contracts as enforcement 3

2.3 MVP rationale and scope 3

3 System Architecture **4**

3.1 Overview 4

3.2 Main layers 4

3.3 Separation of on-chain and off-chain data 5

4 Workflow and Design Description **6**

4.1 Application flow 6

4.2 Contract lifecycle 6

4.3 Key design choices 7

5 Implementation Details **8**

5.1 Smart contract (`DealLock.sol`) 8

5.2 Hardhat configuration and tooling 9

5.3 Frontend modules 9

5.4 Test suite (`test/DealLock.ts`) 9

6 Evaluation and Results **11**

6.1 Smart contract correctness 11

6.2 End-to-end application workflow 11

6.3 Error path coverage 11

6.4 Demo readiness and practical usability 11

7 Limitations and Future Work **12**

7.1 Limitations of the current MVP 12

7.2 Future work 12

8 Member Contribution Statement **13**

1 Introduction

Freelance digital work is often paid in a lump sum at project completion, which creates an asymmetric risk that benefits neither party. The client risks paying before delivery; the provider risks delivering without payment. Milestone payments reduce this exposure by splitting a project into phases, but informal milestone schedules in chats or shared documents remain hard to enforce.

Smart contracts offer a different enforcement mechanism. A self-executing program on a public blockchain can hold funds in escrow, release them only when a defined condition is met, and refund them if a deadline passes without action. Unlike a paper agreement, the enforcement logic is automatic and does not require either party to trust the other with custody of the funds.

DealLock applies this mechanism directly to digital freelance agreements. Each agreement between a client and a provider is a separately deployed Solidity contract instance. The contract stores the milestone list, the escrowed funds, and the recorded delivery references. The React frontend lets either party connect MetaMask, deploy or load a contract, and drive the workflow without a backend server.

The project was conceived as an MVP demonstration suitable for the Sepolia public Ethereum testnet. It targets a focused range of deliverable types: website and landing page development, UI/UX design, and branding or logo packages — service relationships where handoffs are discrete, evidence is linkable, and the workflow phases are easy to specify in advance. This narrower framing is important because the contract enforces workflow and fund movement, not subjective judgements about design quality or scope changes.

2 Background

2.1 Escrow in digital service delivery

An escrow arrangement places funds under the custody of a neutral party or mechanism until a condition is satisfied. In traditional digital freelance platforms such as Upwork or Fiverr, the platform itself acts as the escrow agent: money is deposited before work starts and released when the client marks the order complete. This model works but depends on the platform's rules, custody, fees, and dispute process.

A milestone structure distributes risk further: the provider knows funds are locked before work begins, while the client releases only the tranche attached to a submitted and accepted deliverable.

2.2 Smart contracts as enforcement

Ethereum's programmable account model allows deploying bytecode to a deterministic address on the blockchain. A smart contract executes exactly as written; there is no way for either party — or any third party, including the original deployer — to alter the execution path once the code is deployed. State changes (fund transfers, milestone approvals) are recorded permanently in the chain, are visible to both parties via a block explorer, and require the designated signer's private key. This makes smart contracts a useful primitive for escrow logic, with one important trade-off: the code can enforce state transitions and payments, but it cannot judge whether the delivered work is satisfactory.

2.3 MVP rationale and scope

The project therefore focuses on the smallest complete escrow loop: staged payments, delivery reference recording, client approval, and refund after deadline expiry. Avoiding a backend keeps the trust model simple: binding terms are on-chain, while non-binding titles and notes remain in the browser.

3 System Architecture

3.1 Overview

DealLock is a client-only decentralised application (dApp). Figure 1 outlines the two main layers: the user's browser running the React application, and the Ethereum network hosting deployed contract instances.

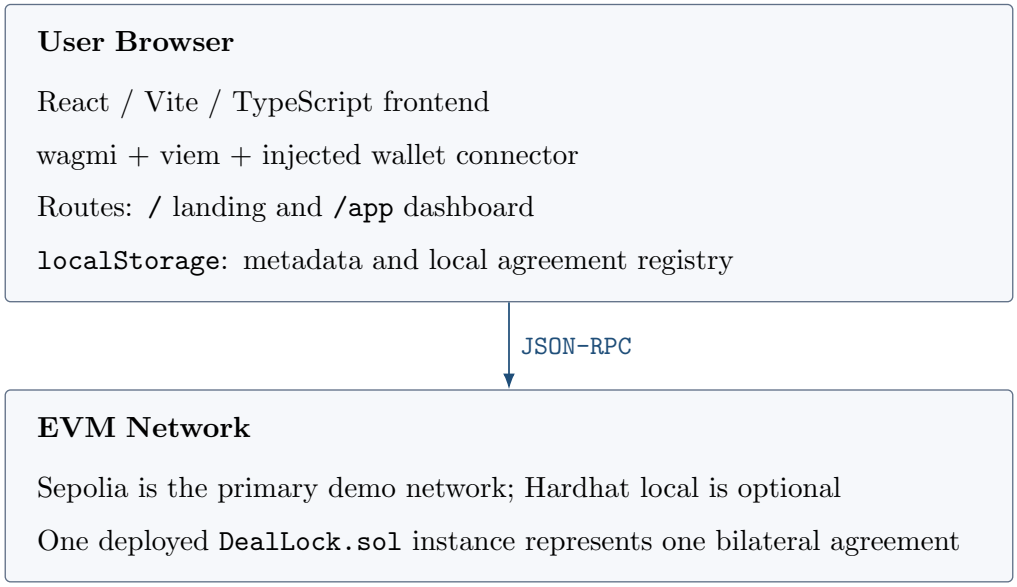


Figure 1: High-level system architecture.

There is no backend, no application server, no database, and no factory contract. Each deployment of `DealLock.sol` represents exactly one agreement between one client and one provider. The only off-chain persistence is the browser's `localStorage` API, which holds optional agreement metadata (project title, service template, guidance notes) and a browser-local registry of contracts the user has created or loaded.

3.2 Main layers

The smart contract is Solidity `^0.8.20` compiled by Hardhat 3 with the optimizer enabled. Construction fixes the client as the deployer, stores the provider address, and writes parallel arrays of milestone descriptions, amounts, and deadlines. These terms are not editable after deployment.

The frontend is a React 19, TypeScript, Vite 6 single-page app using wagmi 2, viem, and TanStack Query. Wallet connection uses the injected provider (MetaMask in the demo). Sepolia is listed first in `wagmi.ts`; Hardhat local remains available for development through the Vite `/rpc-hardhat` proxy. Mainnet is intentionally omitted.

3.3 Separation of on-chain and off-chain data

The design draws a clear line between what is stored on-chain and what is stored in the browser:

On-chain (DealLock.sol)	**Browser only (localStorage)**
Client and provider addresses	Project title
Total amount, funded/released/refunded amounts	Service template choice
Agreement state	Evidence / acceptance guidance notes
Milestone descriptions, amounts, deadlines	Per-milestone notes
Milestone state, proof string, submission timestamp	Agreement registry (addresses seen, sort order)
	Sidebar role filter setting

Table 1: Data separation between on-chain contract state and browser-local storage.

The contract is the canonical source of truth for all binding agreement terms. The `localStorage` data is advisory and UI-enhancement only; it is not shared across browsers or devices, and losing it does not affect the ability to operate the agreement through a different browser.

4 Workflow and Design Description

4.1 Application flow

The frontend has two routes: `/` for the landing page and `/app` for the working dashboard. The dashboard has a browser-local "My agreements" sidebar and a main workspace. With no contract selected, the workspace offers Create and Load modes; with a contract selected, it shows the role, state, parties, ETH totals, available actions, milestone timeline, and saved context.

Creating an agreement is gated by wallet connection and supported-chain checks. The form collects:

- a service template and project title, stored only as local metadata;
- The provider's Ethereum address.
- per-milestone description, ETH amount, future deadline, and optional note.

The client-side checks mirror constructor constraints: valid provider address, at least one milestone, positive amounts, future deadlines, and non-decreasing deadline order. Deployment uses the browser wallet to submit contract creation bytecode; the UI then records the new address and metadata in `localStorage`. Loading an existing agreement only requires a contract address. The dashboard validates the address, stores it in the local registry, and reads party, state, balance, and milestone data from the chain. Wrong-network and wrong-address cases are surfaced as dashboard banners.

4.2 Contract lifecycle

The on-chain lifecycle is:

1. **Proposed**: contract is deployed; provider has not accepted.
2. **Accepted**: provider accepted; client can fund exact total.
3. **Funded**: escrow is active; provider submits, client approves or refunds.
4. **Completed**: all milestone amounts are released or refunded.

The enum also contains `Cancelled` for future extension, but the MVP does not implement cancellation logic. Milestones are sequential: submitting or approving milestone i requires all earlier milestones to be Released, while refund of milestone i requires all earlier

milestones to be resolved (Released or Refunded). The action panel reflects those rules by showing only the buttons the connected role can validly sign.

4.3 Key design choices

The key design choices follow from the MVP goal. One contract per agreement keeps deployment and loading simple. Sepolia gives real wallet signing and public Etherscan inspection without real funds. A backend is avoided so the trust model stays clear: the contract is authoritative, and `localStorage` is only a convenience layer for titles, templates, notes, filters, and agreement history.

5 Implementation Details

5.1 Smart contract (`DealLock.sol`)

The contract defines two state machines: agreement state (`Proposed`, `Accepted`, `Funded`, `Completed`, `Cancelled`) and milestone state (`Pending`, `Submitted`, `Released`, `Refunded`). Each milestone stores description, amount, deadline, proof string, submission time, and state. Agreement-level storage tracks funded, released, and refunded totals, while `client`, `provider`, and `totalAmount` are immutable.

```
enum AgreementState {
    Proposed,
    Accepted,
    Funded,
    Completed,
    Cancelled
}

enum MilestoneState {
    Pending,
    Submitted,
    Released,
    Refunded
}

function fundAgreement()
    external
    payable
    onlyClient
    inState(AgreementState.Accepted)
{
    require(msg.value == totalAmount, "Incorrect funding amount");
    fundedAmount = msg.value;
    agreementState = AgreementState.Funded;
    emit AgreementFunded(client, msg.value);
}

function approveMilestone(uint256 milestoneId)
    external
    onlyClient
    inState(AgreementState.Funded);
```

Listing 1: Core state model and entry points.

The constructor rejects a zero provider address, empty or mismatched milestone arrays, empty descriptions, zero amounts, past deadlines, and unsorted deadlines. Runtime access control is handled by `onlyClient`, `onlyProvider`, and `inState`. The contract also

rejects direct ETH transfers through `receive` and `fallback`, so funds enter only through `fundAgreement`. Approval and refund update state before transferring ETH with low-level `call`; completion is detected when released plus refunded amounts equal `totalAmount`. Events record acceptance, funding, submission, release, and refund.

5.2 Hardhat configuration and tooling

`hardhat.config.ts` uses the Hardhat 3 Mocha/Ethers toolbox, Solidity 0.8.20, optimizer runs set to 200, and tests under `test/`. Sepolia is configured only when `SEPOLIA_RPC_URL` and `PRIVATE_KEY` are present, so local tests do not require network credentials.

5.3 Frontend modules

`main.tsx` installs the wagmi and React Query providers and routes `/` to `LandingPage` and `/app` to `DashboardPage`. `wagmi.ts` defines Sepolia-first transports, while `network.ts` centralises chain IDs, explorer links, and `switchToSepolia`. The ABI and browser deployment bytecode are exported from `contract.ts` and `bytecode.ts`; the latter must be regenerated after contract changes.

The dashboard reads scalar contract fields with `useReadContract` and milestones with `useReadContracts`, gated by wallet connection, supported chain, and a valid contract address. `storage.ts` implements two browser-only persistence layers: per-address metadata and a local agreement registry. `types.ts` provides role derivation by comparing the connected wallet with on-chain client/provider addresses. `templates.ts` supplies off-chain service templates; they prefill UI guidance but do not change constructor arguments.

The main interaction components cover creation, actions, milestones, overview, and the agreement sidebar. They expose only actions valid for the connected role and contract state: provider accept/submit, client fund/approve/refund, or read-only display for unrelated wallets. Transaction states are shown as pending, confirming, success, or error, with Sepolia Etherscan links where available.

5.4 Test suite (`test/DealLock.ts`)

The test suite uses Mocha and the Hardhat 3 network helpers with the ethers plugin. Tests run against a local in-memory Hardhat chain only; no network credentials are required. The fixture `deployDealLockFixture` deploys a three-milestone agreement (Prototype, Beta, Final at 1/2/3 ETH with deadlines 100/200/300 seconds into the future) using three signers: `client`, `provider`, and `other`.

The eleven tests cover deployment state, provider-only acceptance, exact funding, submission after funding, sequential submission, approval payout, rejection of unsubmitted approval, expired refunds, early-refund rejection, sequential refund ordering, and full completion. They assert revert reasons, emitted events, and Ether balance changes for release/refund paths.

6 Evaluation and Results

6.1 Smart contract correctness

The Hardhat test suite was run against the final version of the contract. All eleven tests pass. The suite exercises the reachable state transitions and access-control boundaries: only the provider can accept, only the client can fund/approve/refund, funding must match `totalAmount`, milestones cannot be submitted out of order, approval transfers the correct amount to the provider, refunds are allowed only after deadline expiry, and completion is reached after all milestones are released. No unexpected reverts or state inconsistencies were observed.

6.2 End-to-end application workflow

The repository also contains a Sepolia runbook for a two-wallet demo: client deploys, provider accepts, client funds, provider submits evidence, and client approves or refunds until completion. The frontend follows this lifecycle and adds Etherscan links for Sepolia transaction hashes and addresses. The report does not present performance numbers because no benchmark data was collected.

6.3 Error path coverage

The frontend handles common demo failures: unsupported chains trigger a switch prompt; wallet addresses or wrong-chain contract addresses produce a readable "no DealLock data" message; missing Hardhat local RPC returns an explanatory JSON-RPC error; and wallets that are neither client nor provider see a read-only state instead of write buttons.

6.4 Demo readiness and practical usability

The application is suitable for a live classroom demo. The sidebar supports quick switching between agreements, the header shows role/state/ETH progress, and pending/confirming/success/error states make public-testnet delays understandable.

7 Limitations and Future Work

7.1 Limitations of the current MVP

DealLock's limitations come directly from the MVP scope. The local agreement registry and metadata are browser-only, so users must keep the contract address if they switch devices or clear browser storage. The contract has no arbitration or proof-quality oracle: it checks that a proof string exists, but not whether a link works or the delivered work is good. It supports native ETH only, not ERC-20 tokens or stablecoins. There is also no factory, marketplace, indexer, global agreement search, or on-chain revision workflow; milestone terms are fixed at deployment. Finally, one deployment per agreement is easy to understand but would be inefficient for a production marketplace with many agreements.

7.2 Future work

Useful extensions would be an event indexer or factory contract for agreement discovery, optional mediator-based dispute resolution, ERC-20 or stablecoin payments, cross-device metadata sync through wallet-signed writes, and a more structured proof format such as URL plus content hash. These additions would move DealLock from a classroom escrow MVP toward a more practical product while preserving the same core state machine.

8 Member Contribution Statement

The project was completed by a group of two students, as registered on the FTE4312 project proposal. The following table summarises each member's primary areas of responsibility.

Member	**Contributions**
Ho Dang Dung (325040027)	Smart contract design and implementation (`DealLock.sol`); Hardhat configuration; Mocha test suite; CLI deploy script; contract ABI and bytecode wiring for the frontend.
Taisiia Ibragimova (325040033)	Core frontend architecture; wagmi and viem integration; dashboard page and component structure; wallet panel; agreement overview; action panel; transaction state management; storage layer and agreement registry; service templates and off-chain metadata schema; landing page; sidebar and registry UI; CSS design system; documentation (`README.md`, `docs/mvp.md`, `description.txt`).

Table 2: Summary of member contributions.

Both members participated in the specification review, end-to-end demo testing, and preparation of this final report. The workload was distributed approximately equally between the two members.